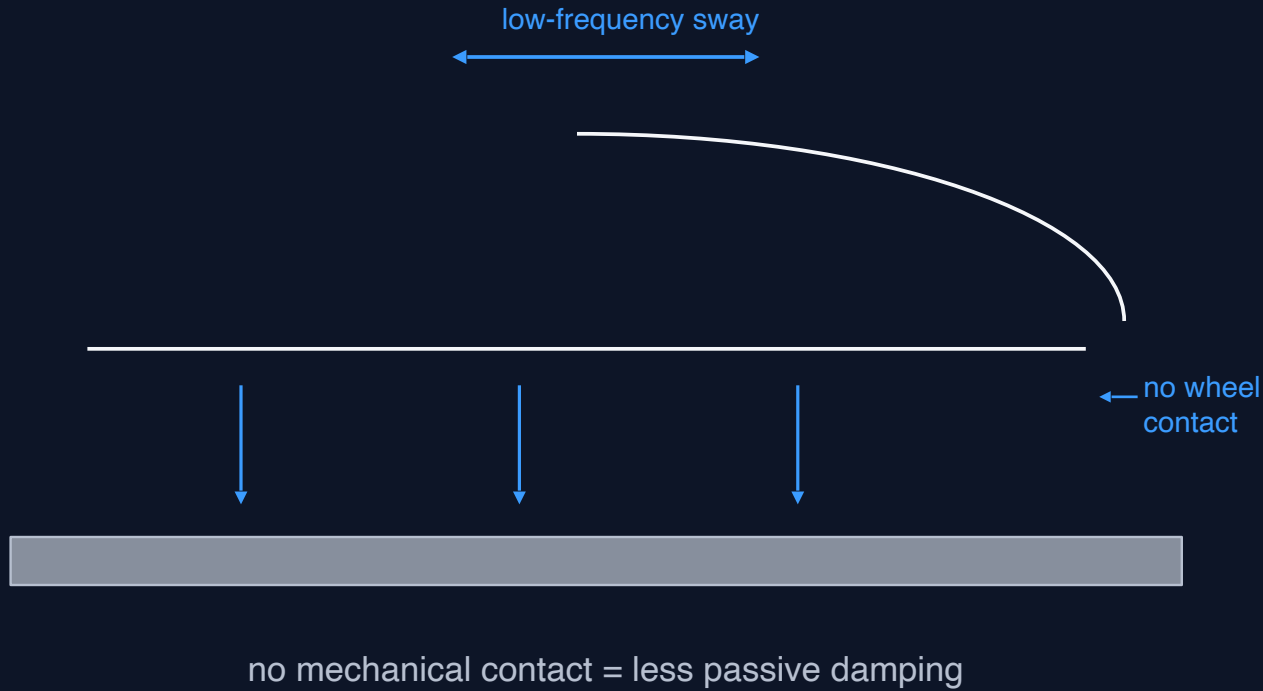


Vibration Control

Adaptive ML control for electromagnetic vibration damping in maglev/hyperloop systems

Sripaadh Jayashree Kuppusamy & Ishita Sharma

The problem: vibration without contact



Why it matters

- Less friction, but less natural damping
- Low-frequency sway affects passenger comfort
- Larger oscillations can become safety risks
- Active damping needs real-time control

Our project studies a scaled electromagnetic damping approach for this control problem.

What existed before vsHacks / What we built

Research foundation

- physical electromagnet prototype
- PWM-to-field measurements
- spatial field measurements
- physics-based damping model

Built during vsHacks

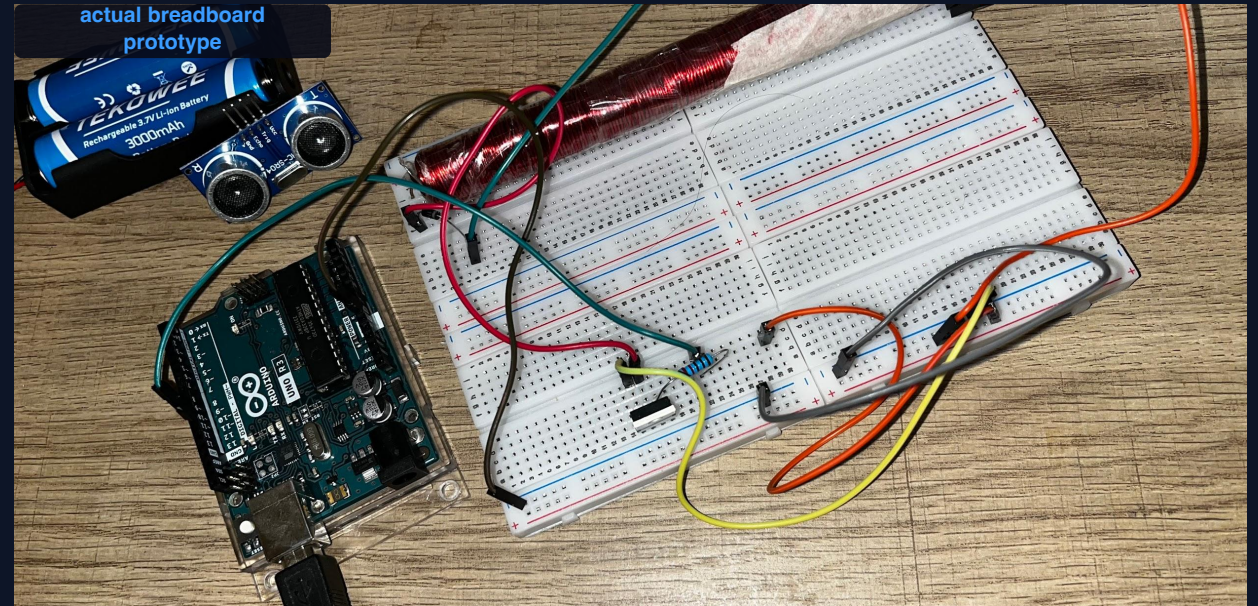
- adaptive gain-selection model
- vibration classifier
- integrated sense-classify-adapt pipeline
- Arduino serial bridge
- open-source demo

The hardware provided the measured physics; during vsHacks, we built the complete adaptive intelligence and software pipeline around it.

Hardware architecture

- 1 HC-SR04 ultrasonic sensor measures displacement
- 2 Arduino Uno runs a proportional-derivative control loop
- 3 PWM signal sets the duty cycle
- 4 MOSFET driver switches current through the coil
- 5 Soft-iron electromagnet creates the magnetic field
- 6 Conductive rail eddy currents oppose motion

closed loop: sense → compute → actuate → damp



**MOSFET
current control**



**Flyback diode
inductive kickback
protection**

These images come from the physical prototype and circuit components used in the damping setup.

The core problem: nonlinear actuation

Measured equation

$$B = 0.0644 \cdot e^{(0.00706 \cdot OCR)} + 7.6024$$
$$R^2 = 0.975$$

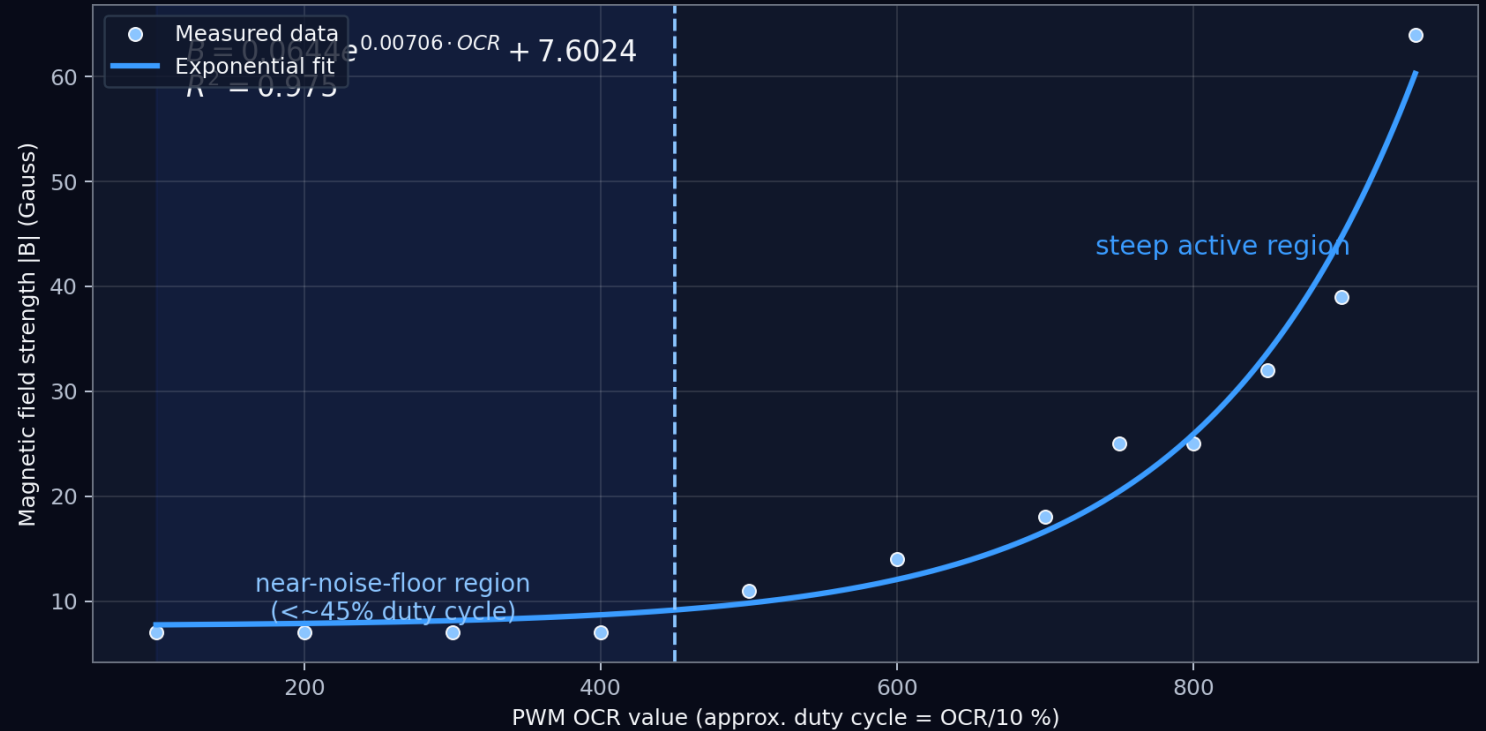
Damping force scales approximately with the square of magnetic field strength:

$$F \propto B^2$$

Why it matters

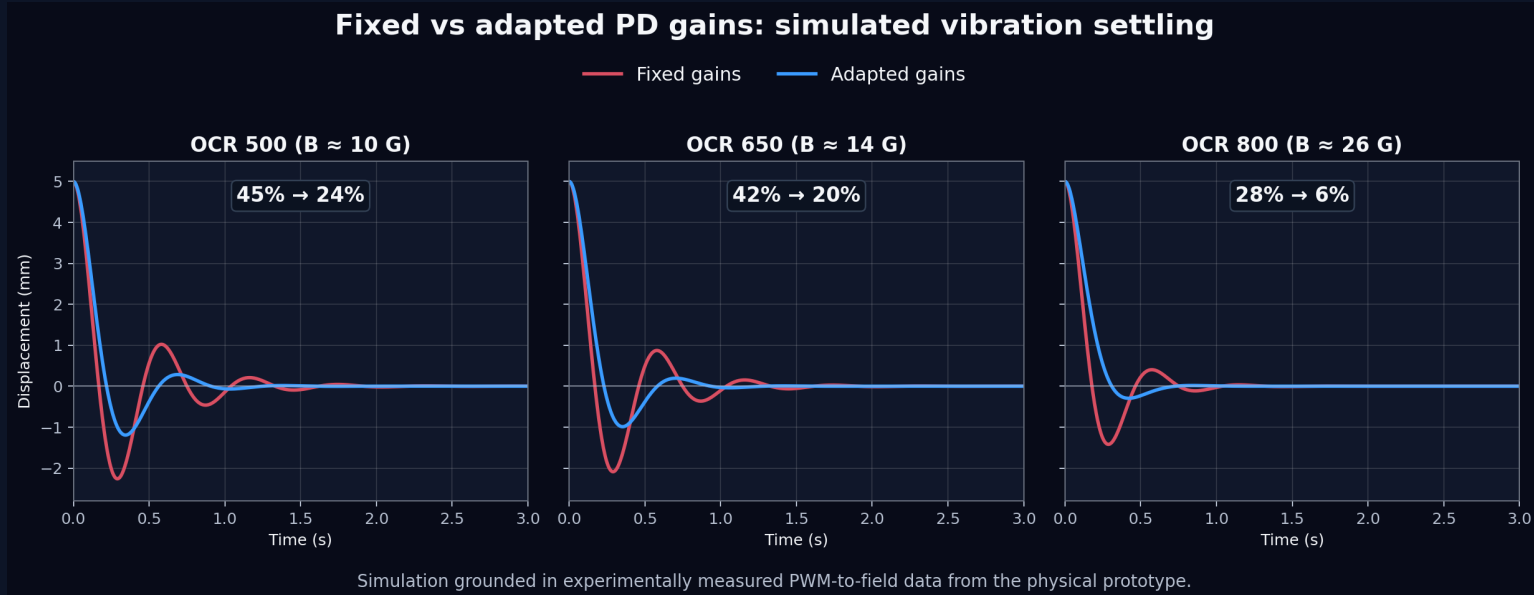
- Near-noise-floor below ~45% duty cycle
- Steep active region above it
- One fixed gain set performs inconsistently in simulation

Measured PWM-to-field relationship



The ML layer is motivated by measured physics, not added arbitrarily.

Adaptive gains reduce simulated overshoot



Summary

- Fixed gains tuned at full field
- OCR 500: 45% → 24%
- OCR 650: 42% → 20%
- OCR 800: 28% → 6%
- Both controllers settle to zero

This is simulation, not hardware-in-the-loop yet.

Simulation is grounded in experimentally measured PWM-to-field data from the physical prototype.

02

ML Implementation

The code adds two lightweight Random Forest models on top of the characterized actuator.

Model selection: two ML components

Pathway 1: Adaptive gain model

Predicts the best-performing K_p and K_d found within the simulation search grid for the current operating point.

Inputs

- OCR value
- estimated magnetic field
- normalized control authority

Outputs

- K_p
- K_d

Labels come from a grid search over the mass–spring–damper simulation.

Pathway 2: Vibration classifier

Classifies a window of displacement readings into a disturbance type using time-domain and FFT features.

Inputs

- displacement window
- time-domain features
- FFT features

Outputs

- no disturbance
- low-frequency sway
- high-frequency vibration
- impulse shock

Random Forests were chosen for fast inference, interpretability, and reliability on small synthetic/simulation datasets.

Code: from sensor window to disturbance label

1 Extract interpretable signal features

src/vibration_classifier.py

```
def extract_features(window):
    w = window - np.mean(window)

    rng_pp = np.ptp(w)
    rms = np.sqrt(np.mean(w ** 2))
    spike = np.max(np.abs(w)) / (rms + 1e-9)

    spec = np.abs(rfft(w))
    freqs = rfftfreq(WINDOW, 1 / FS)
    dom_freq = freqs[np.argmax(spec)]

    low_band = np.sum(spec[freqs < 2.0]) / (np.sum(spec)
+1e-9)
    high_band = np.sum(spec[freqs >= 2.0]) / (np.sum(spec)
+1e-9)

    return [rng_pp, rms, spike, dom_freq, low_band,
high_band]
```

Each displacement window becomes time-domain + FFT features that are easy to explain in the demo.

2 Classify the disturbance type

src/pipeline.py

```
feats = extract_features(window)
cls_idx = int(clf.predict([feats])[0])
cls = CLASS_NAMES[cls_idx]

CLASS_NAMES = [
    "no_disturbance",
    "low_freq_sway",
    "high_freq_vibration",
    "impulse_shock"
]
```

- A rolling sensor window is converted into features.
- A Random Forest predicts the disturbance class.
- The class is used later to bias the control response.

Takeaway: the classifier makes its decision using clear vibration features such as signal strength, frequency, and sudden peaks.

Code: measured physics —> adaptive gains

1 Physics functions

src/gain_model.py

```
def pwm_to_field(ocr):  
    return 0.0644 * np.exp(0.00706 *  
ocr) + 7.6024  
  
def control_authority(B):  
    return 0.2 + 0.8 * (B**2 -  
B_MIN**2) / \  
        (B_MAX**2 - B_MIN**2)
```

The model input is grounded in the measured PWM → magnetic-field curve, then converted into normalized control authority.

Input features

- OCR value
- Estimated field strength B

2 Class-specific bias

src/pipeline.py

```
CLASS_BIAS = {  
    "no_disturbance":      (0.5, 0.5),  
    "low_freq_sway":      (1.2, 0.9),  
    "high_freq_vibration": (0.9, 1.2),  
    "impulse_shock":      (1.0, 1.4)  
}
```

This part is deliberately interpretable: sway favors proportional response, while shocks lean more on derivative response.

The ML predicts base gains, then transparent rules tune the response for each disturbance type.

3 One full control step

src/pipeline.py

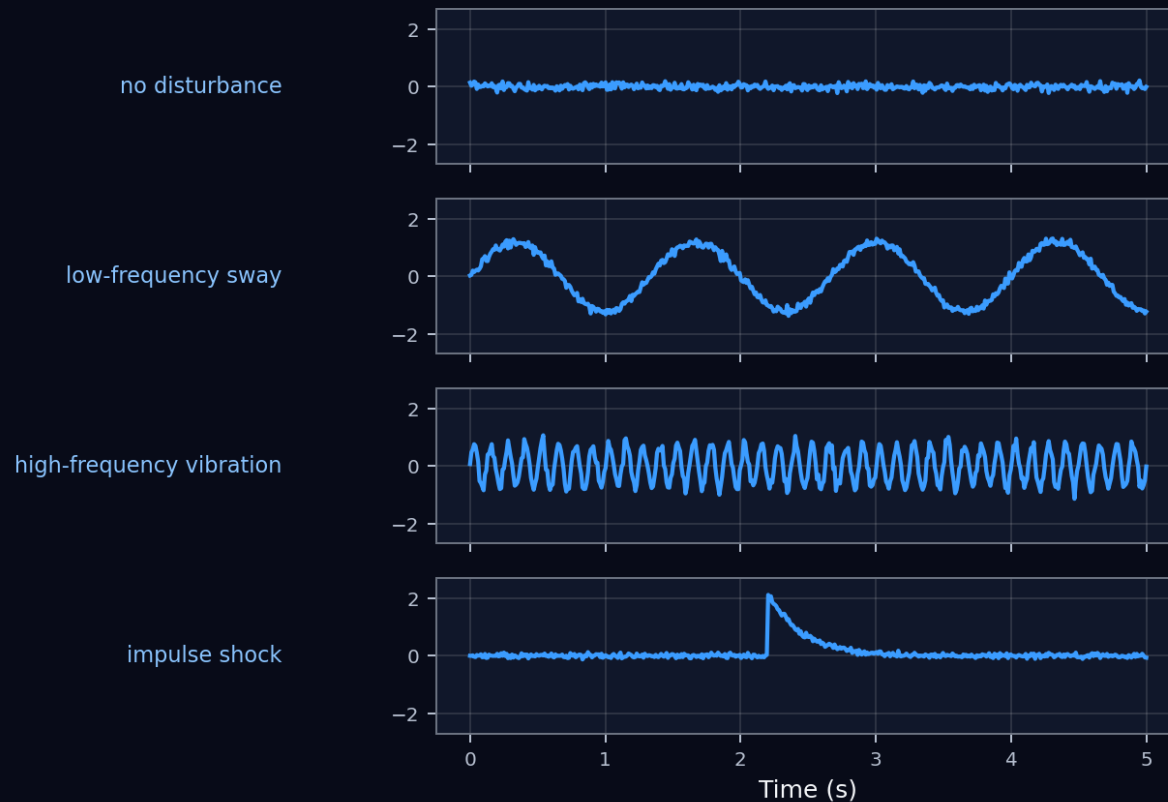
```
B = pwm_to_field(ocr)  
auth = control_authority(B)  
  
base_kp, base_kd = gain_model.predict([[ocr,  
B, auth]])[0]  
  
bp, bd = CLASS_BIAS[cls]  
kp, kd = base_kp * bp, base_kd * bd  
  
return {  
    "class": cls,  
    "field_G": round(float(B), 1),  
    "final_gains": (round(float(kp), 1),  
round(float(kd), 1))  
}
```

This is the demoable loop: sense → classify → estimate field → select gains → adjust control.

Takeaway: the code links physical measurement, ML classification, and adaptive control in one executable pipeline.

Training data and preprocessing

Synthetic waveform classes for classifier training



Extracted features

- RMS
- peak-to-peak amplitude
- spikiness
- dominant frequency
- low-frequency energy
- high-frequency energy

Classifier training and evaluation currently use synthetic waveforms.

Synthetic signals make the demo run offline for judging, while live HC-SR04 input can be streamed through the serial bridge.

Integration: sense —> classify —> adapt



Runs offline on synthetic data; an optional serial bridge streams live HC-SR04 displacement data over USB when hardware is connected.

Conclusions

The measured nonlinearity makes one fixed gain set difficult to tune consistently across the full operating range.

Measured

Nonlinear PWM-to-field behavior
in the physical actuator

Demonstrated

Adapted gains reduced
simulated overshoot at several
operating points

Next

Real sensor recordings,
hardware-in-the-loop control, and
direct damping-force
measurement

Results are based on experimentally measured PWM-to-field data and simulations of the experimental prototype.

Thank you!

Questions?

Contact

sripaadhj@gmail.com
ishitabuilds@gmail.com

Demo command

```
python3 maglev-damping-ml/src/pipeline.py
```

GitHub repository

[github.com/IonizedAlphaParticle/
activevibcontrol](https://github.com/IonizedAlphaParticle/activevibcontrol)

Open-source project for vsHacks
2026



Scan for repository